DBMS MINI PROJECT · PROJECT GUIDE

ScholarMesh

Find study notes by meaning — and the classmates studying the same ideas.

Team	Darshan P Pawar (PES2UG24CS143) · Dhanush V Biradar (PES2UG24CS155)
College	PES University, Electronic City Campus · Dept. of CSE
Code	github.com/Darshanpawar7/DBMS-Mini-Project
This guide	What the project is · tech stack · how it works · screenshots · how to run it on your computer

Short on time? Read page 2 (what it is), the stack on page 3, and the 5 setup steps in section 5. The full technical report is `docs/ScholarMesh_Phase2_Report.pdf`.

1. What is ScholarMesh?

A website where students upload their study notes, search them **by meaning** (not just exact words), and get **matched into study groups** with classmates who study the same topics.

The problem

- Notes are scattered across Drive folders, WhatsApp groups and laptops.
- Normal search only finds **exact words**. Searching *"how do models decide which words matter"* finds nothing, even though a note called *"Transformers and Self-Attention"* explains exactly that.
- Finding study partners is random — you ask around in class groups.

What ScholarMesh does

Search by meaning

Ask in your own words. The app understands what you mean and finds the right notes, even with no matching words.

Smart study groups

Suggests groups and classmates whose interests are closest to yours. Seat limits are enforced.

Safe sharing

Each note is *public*, *course-only* or *private*. Nobody ever sees a note they are not allowed to.

Who uses it

Role	What they can do
Student	Upload notes, search, save (bookmark) notes, enroll in courses, create / join / leave study groups, see their dashboard.
Group admin	A student who created or runs a group: edit the group, promote or remove members.
Admin	Platform manager: usage charts, manage users and roles, manage courses and tags, read the audit log.

Why it counts as a DBMS project (not just a CRUD app)

- **Two kinds of database in one:** normal tables (SQL) for users, courses and groups, plus a **vector store** (pgvector) for the meaning of notes.
- **The database enforces the rules itself** using triggers, constraints and transactions — e.g. a full group can never accept another member, even if two students click "Join" at the same moment.
- Uses views, a materialized view, stored functions and procedures, indexes, full-text search, vector search and an audit log.

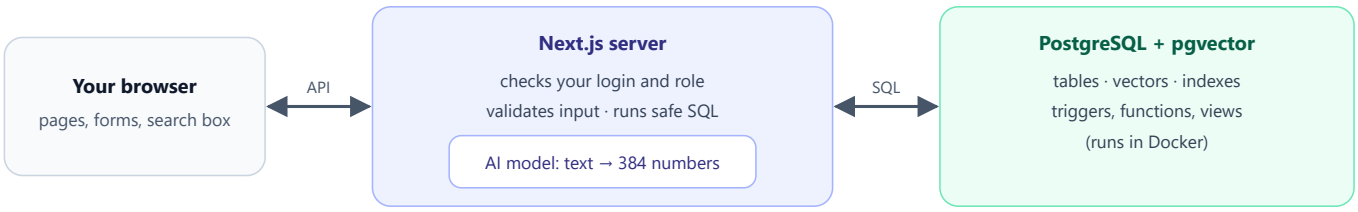
2. Tech stack

Part	Technology	What it does here (in simple words)
Database	**PostgreSQL 17**	Stores everything: users, courses, notes, groups, bookmarks, searches. Checks every rule.
Vector search	**pgvector** (PostgreSQL extension)	Stores each note passage as 384 numbers that capture its meaning, and finds the closest ones fast (HNSW index).
Keyword search	PostgreSQL **full-text search**	Classic word search with word stems ("normalize" ≈ "normalization") and ranking (GIN index).
AI model	**all-MiniLM-L6-v2** via Transformers.js	Turns text into those 384 numbers ("embeddings"). Runs on your own computer — no API key, no cost.
Website + API	**Next.js 16**, React 19, TypeScript	The pages you see and the backend API they call. The browser never talks to the database directly.
Styling	**Tailwind CSS 4**	The look and layout, including phone screens.
Login & security	bcrypt, signed JWT cookie, zod	Passwords stored only as hashes; a secure login cookie; every input checked.
Runs in	**Docker** (Docker Compose)	Starts the database with one command — nothing to install for PostgreSQL itself.
Testing	Vitest, Playwright	101 automated tests: 91 code and database tests + 10 tests that click through the real website.
Version control	Git + GitHub	All code, history and documents.

Why PostgreSQL + pgvector (Option 2) instead of a separate vector database? Because the meaning-search and the "who may see this note?" check run **in the same SQL query**, a private note can never leak into results. Foreign keys also keep the vectors in sync with the notes — delete a note and its vectors disappear in the same transaction.

3. How it works

The big picture



The journey of a note

1. **Upload** — a PDF, TXT or Markdown file (or pasted text). The server pulls out the text.
2. **Save** — in one **transaction**: the note is saved, and a database **trigger** splits the text into short passages (≤ 800 characters). If anything fails, nothing is saved.
3. **Understand** — the AI model turns every passage into 384 numbers (an **embedding**). Passages with similar meaning get similar numbers.
4. **Find** — the note can now be found by its words and by its meaning.

The three search modes

Mode	How it works	Think of it as…
Keyword	Matches the actual words (full-text search, GIN index), ranks by relevance, highlights matches.	Ctrl+F across all notes
Semantic	Turns your question into 384 numbers and finds the passages whose numbers are closest (cosine similarity, HNSW index).	A librarian who understands what you mean
Hybrid (default)	Runs both and merges the two rankings (Reciprocal Rank Fusion), so exact words *and* meaning both count.	The best of both

Who can see which note

Public → every logged-in user · **Course-only** → students enrolled in that course · **Private** → only the owner (and admins). One database function, `fn_visible_doc_ids`, makes this decision for every search, list, download and suggestion.

How study-group matching works

Each student gets an **interest vector**: the average meaning of the notes they uploaded or saved and the questions they searched. A group's interest is the average of its members'. Groups in your courses with a free seat are ranked by how close they are to you (70%) plus how many topic tags you share (30%). Classmates are ranked the same way.

Rules the database enforces by itself

- A group can't go over its seat limit — the group row is locked while someone joins, so two people can't take the last seat.

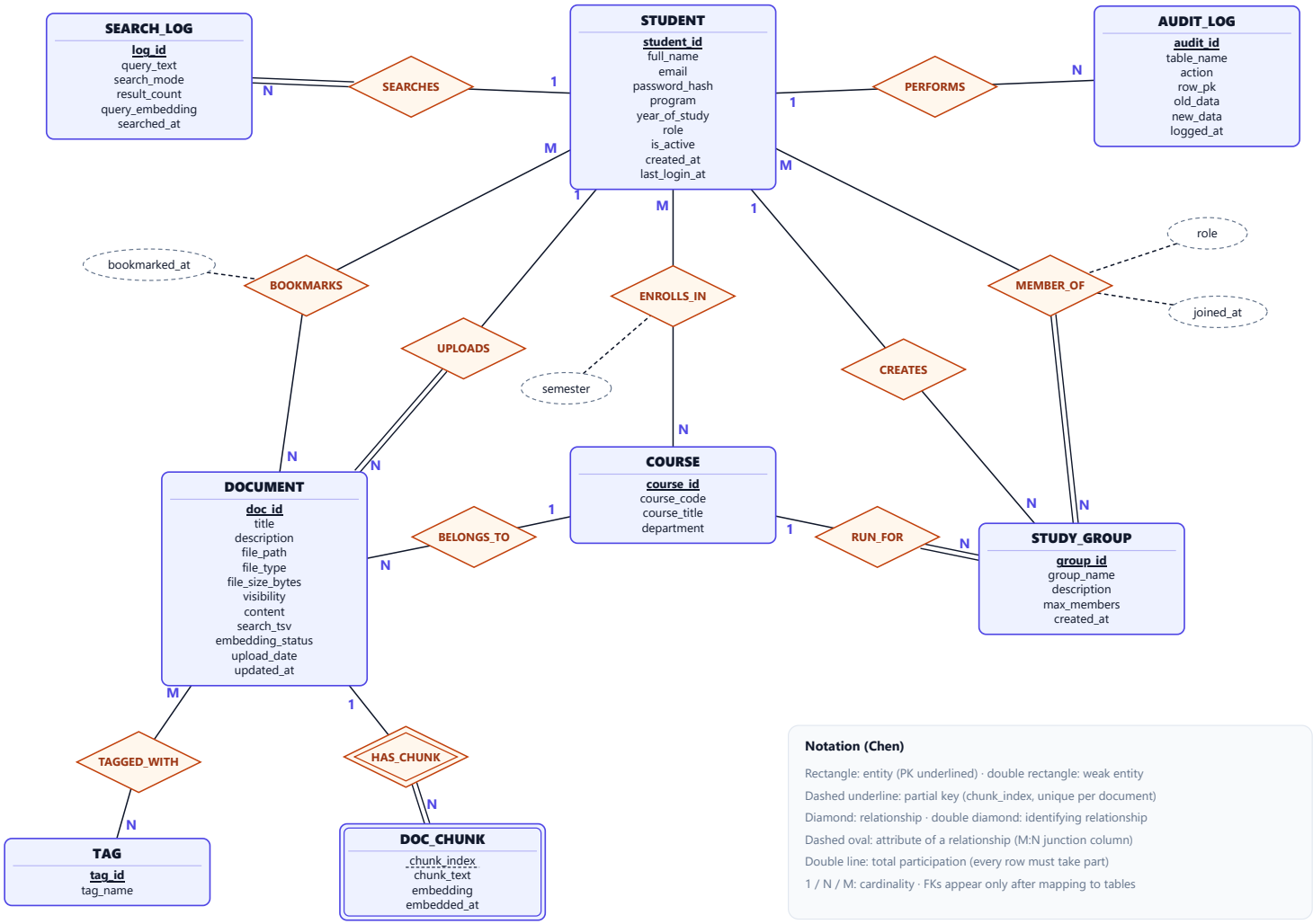
- Only students enrolled in a course can create or join its groups. The creator automatically becomes group admin.
- If the last admin leaves, the longest-standing member becomes admin; an empty group is deleted.
- Dropping a course removes you from its groups. Deleting a course turns its course-only notes private.
- Passwords must be bcrypt hashes (plain text is rejected). E-mails, tags and semesters must be in the right format.
- Every important change (roles, notes, courses, groups) is written to an audit log with who did it.

The database at a glance — 12 tables

Table	Stores
STUDENT	Users (students and admins), password hash, role, active or not
COURSE	Courses, e.g. UE23CS351A Database Management Systems
ENROLLMENT	Which student takes which course in which semester (many-to-many)
DOCUMENT	Notes: title, text, file, visibility, owner, course
DOC_CHUNK	Passages of each note + their 384-number vectors (pgvector)
TAG / DOCUMENT_TAG	Topic labels and which notes have them
BOOKMARK	Notes each student saved
STUDY_GROUP / GROUP_MEMBERSHIP	Groups (per course, with a seat limit) and their members with roles
SEARCH_LOG	Every search (for history, analytics and interest profiles)
AUDIT_LOG	History of important changes (written only by triggers)

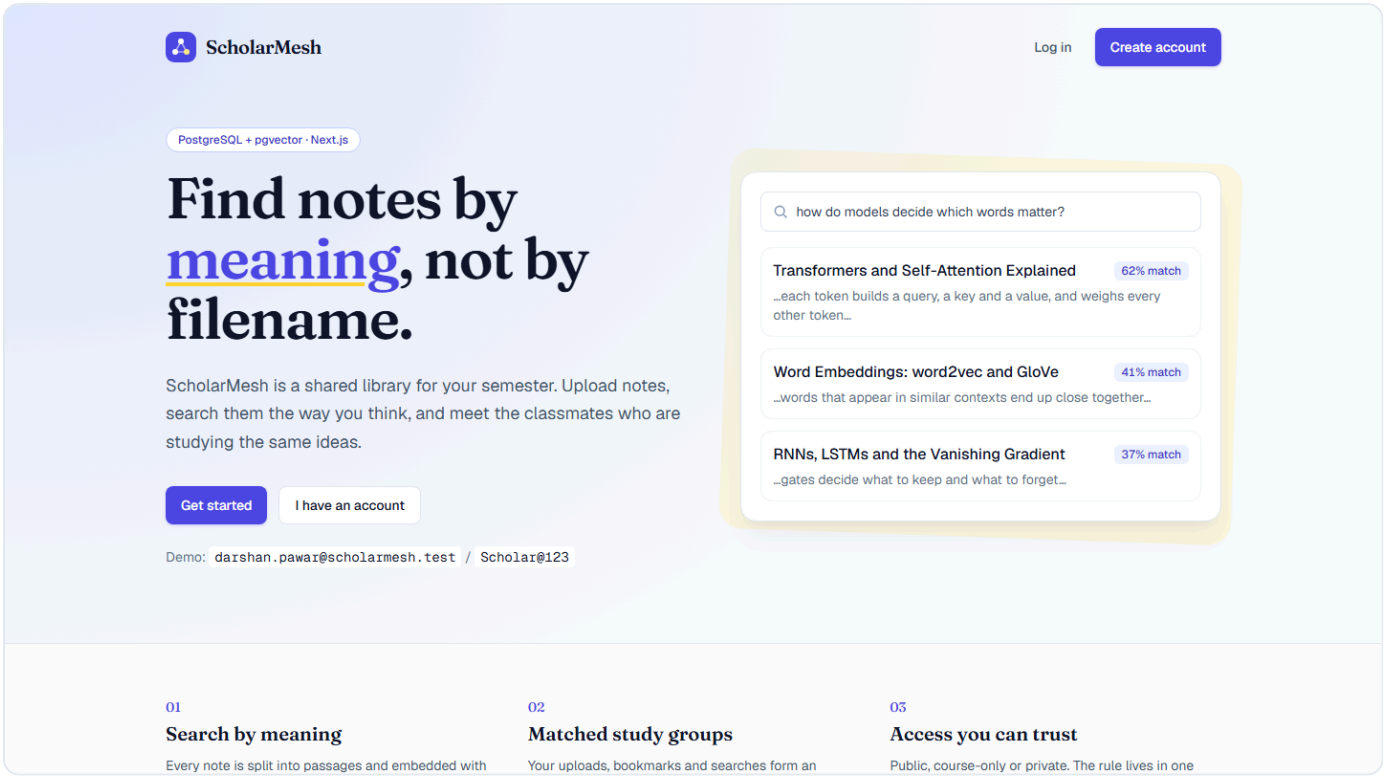
The full ER diagram is on the next page. Column-by-column details are in the technical report (section 9).

The ER diagram

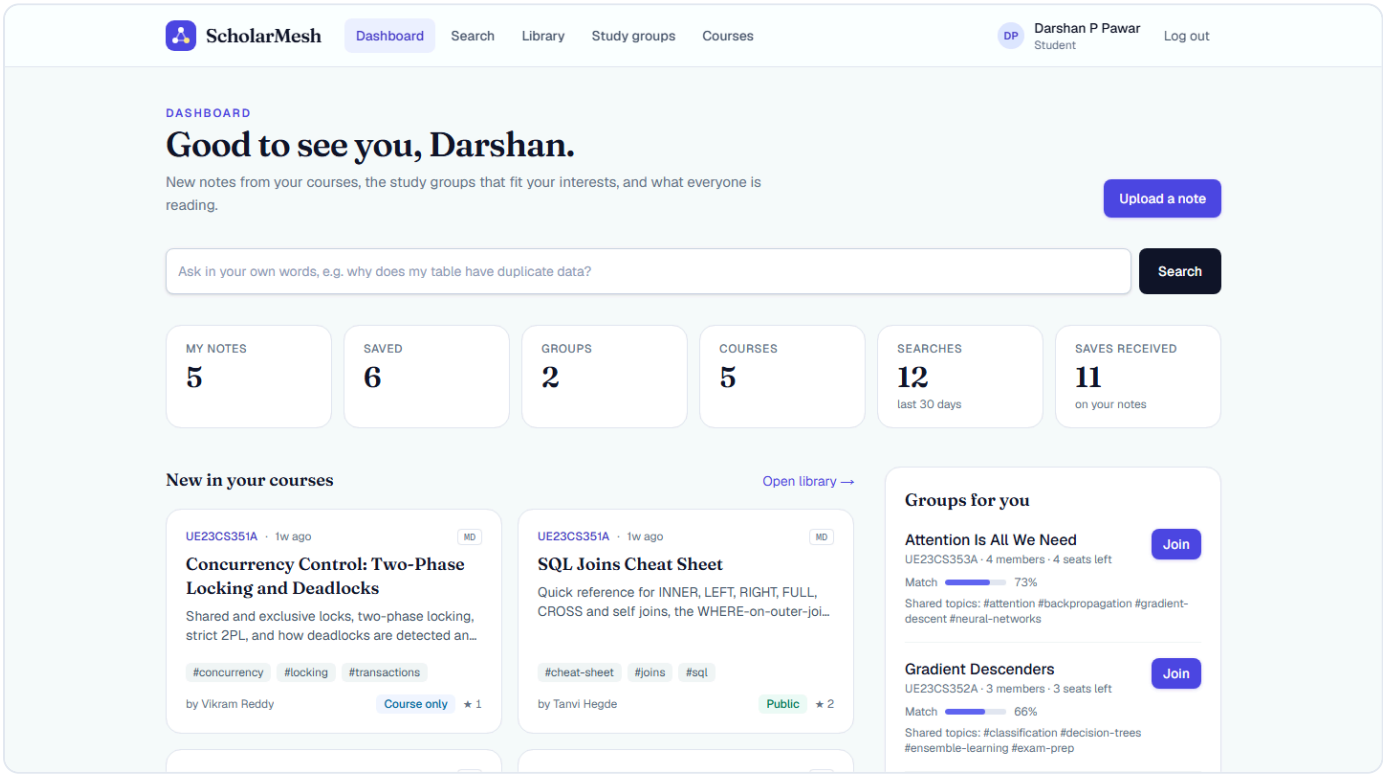


Rectangles = tables (entities), diamonds = relationships, 1 / N / M = how many on each side, double line = must take part, double box = weak entity (a passage belongs to its note).

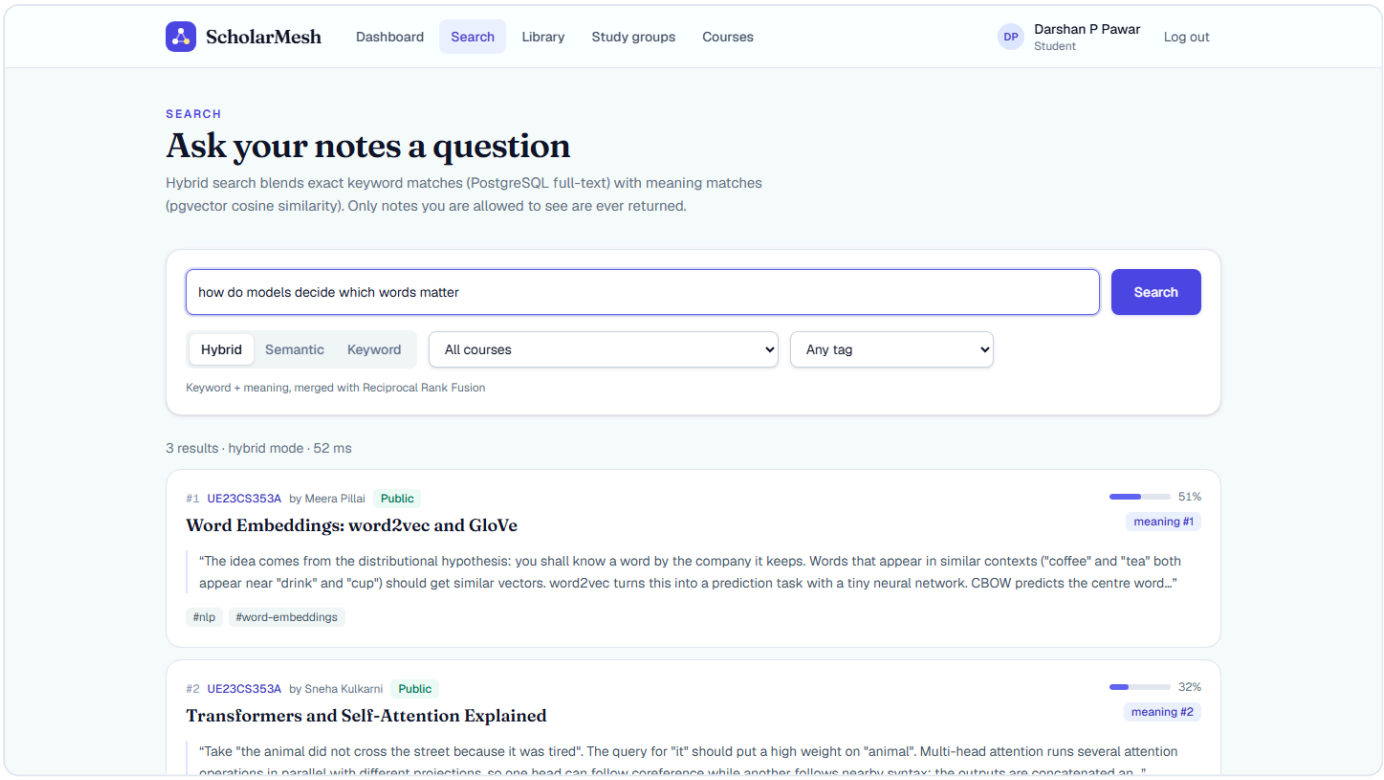
4. Screenshots



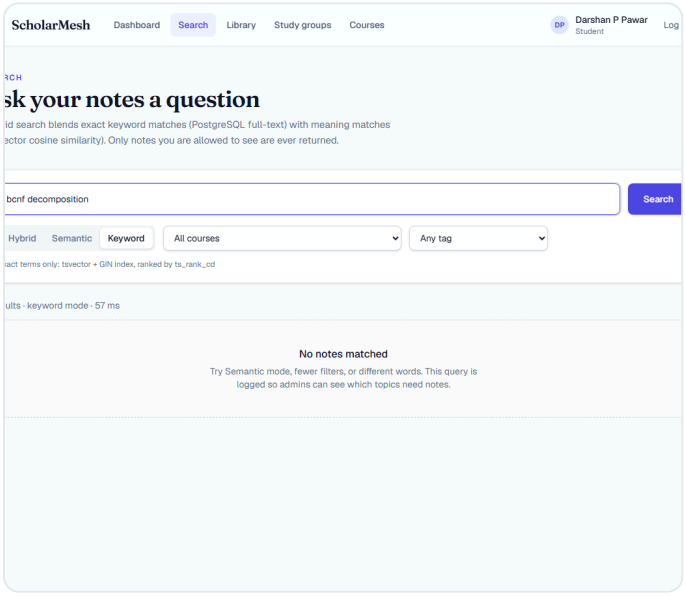
Home page — what the app does, with a demo login.



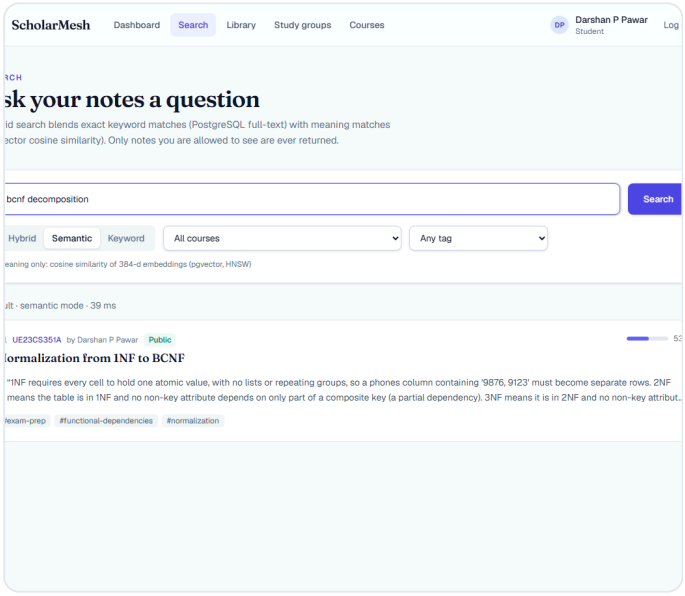
Student dashboard — your numbers, new notes from your courses, suggested groups and classmates, trending topics.



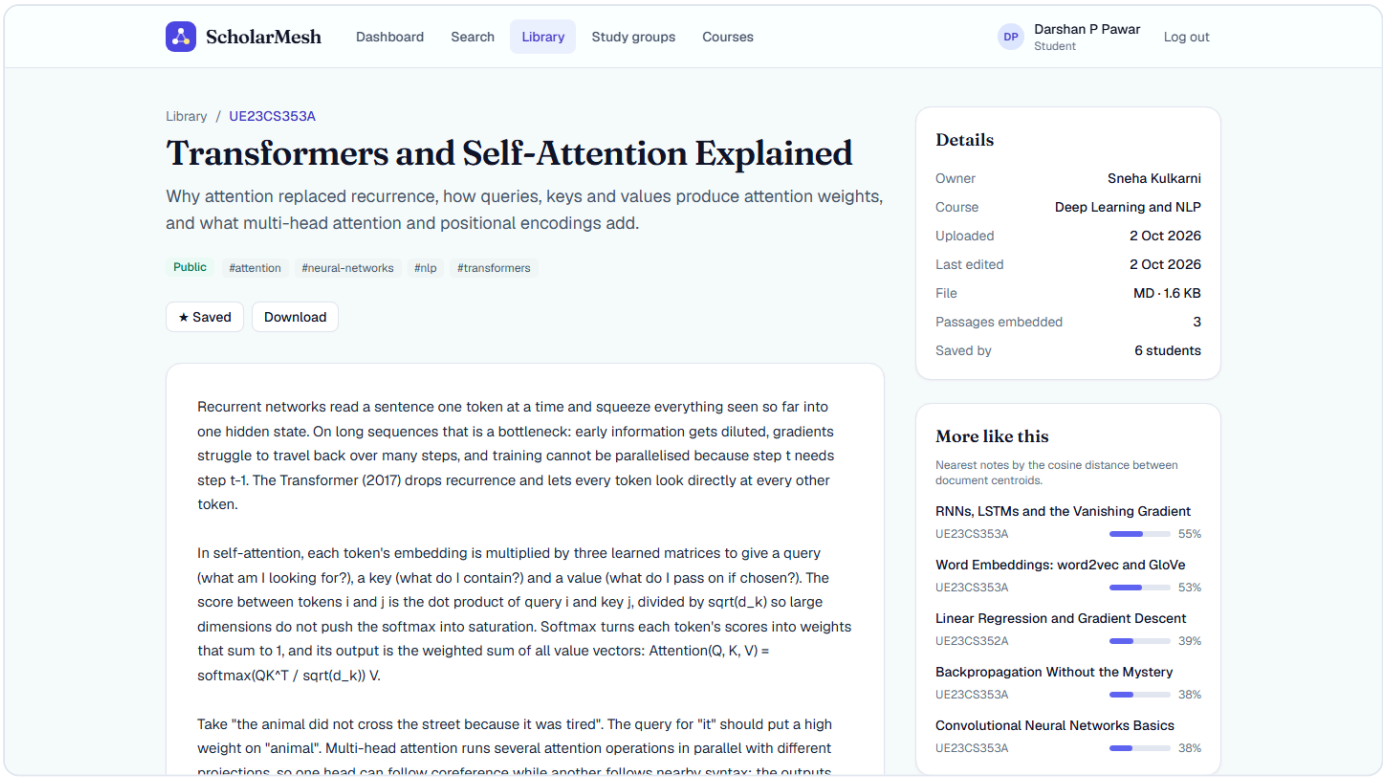
Search by meaning — the question shares no words with the notes, yet the right notes come back with a match score.



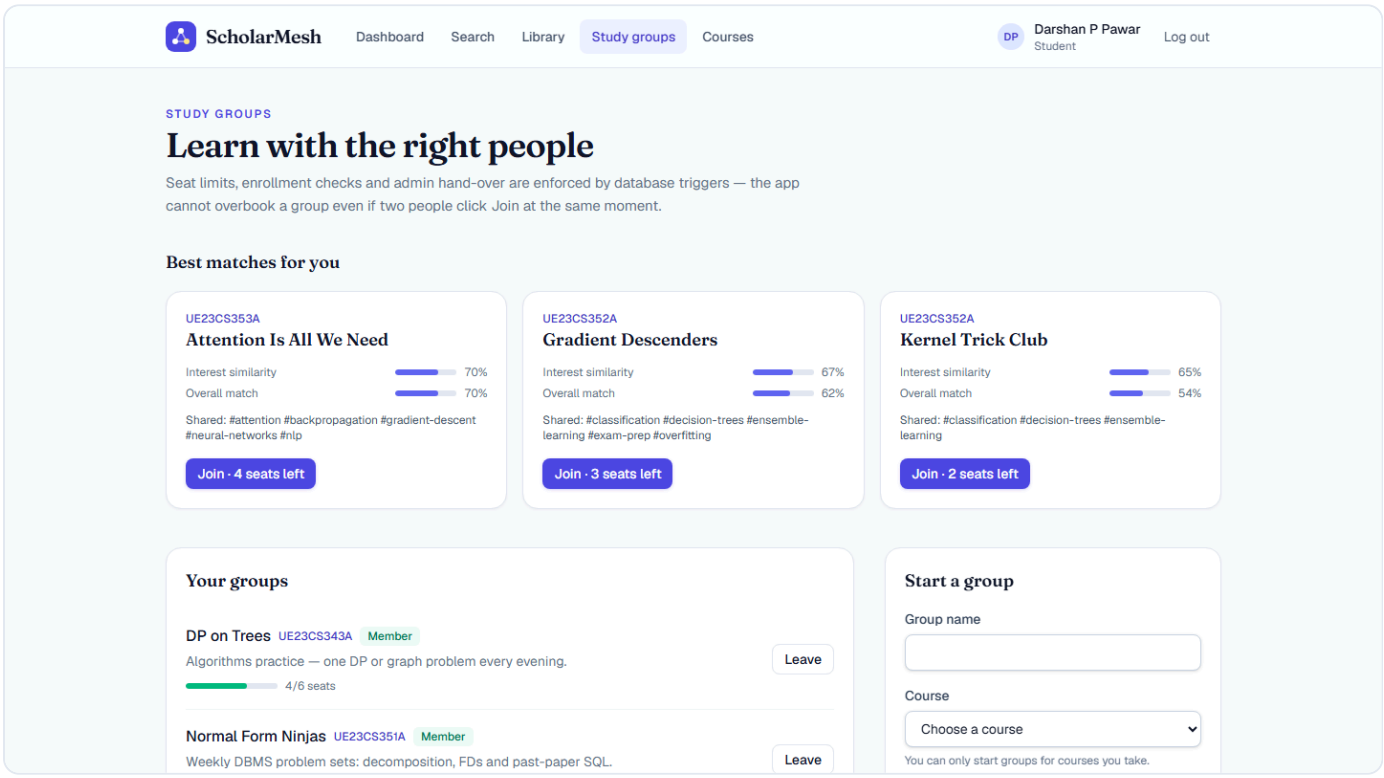
Keyword mode: "bcnf decomposition" → 0 results (no note contains "decomposition").



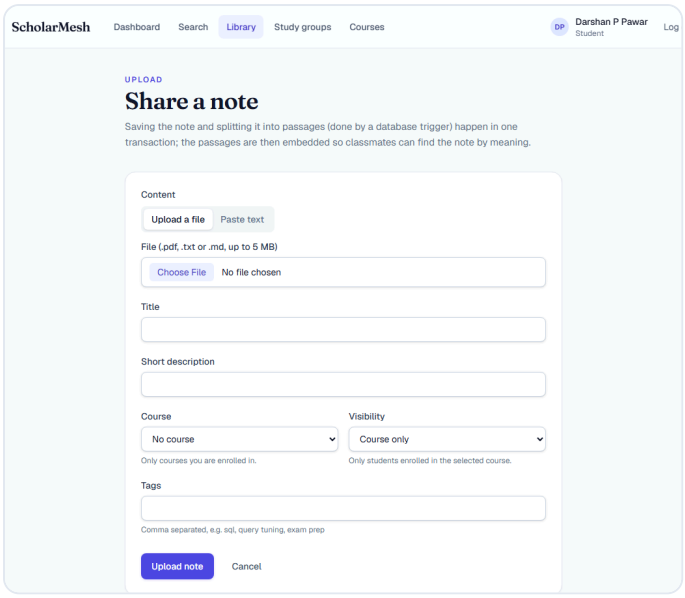
Semantic mode: the same question finds the Normalization note by meaning.



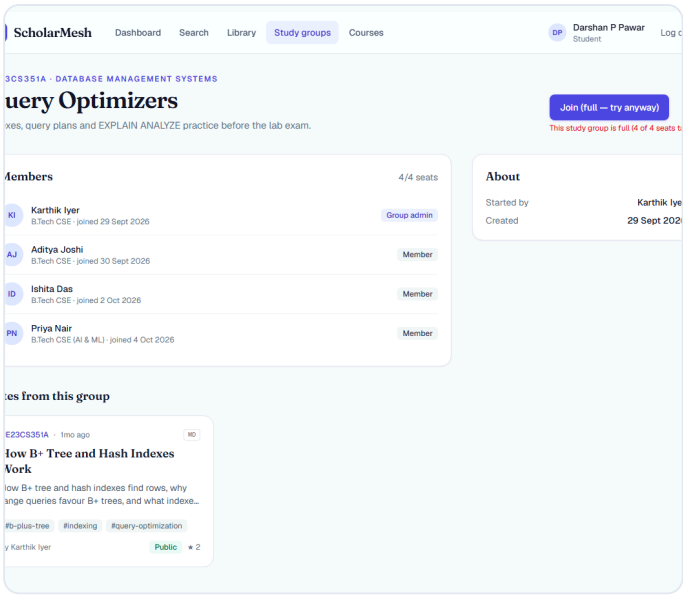
A note — save it, download it, and see "more like this" (similar notes by meaning).



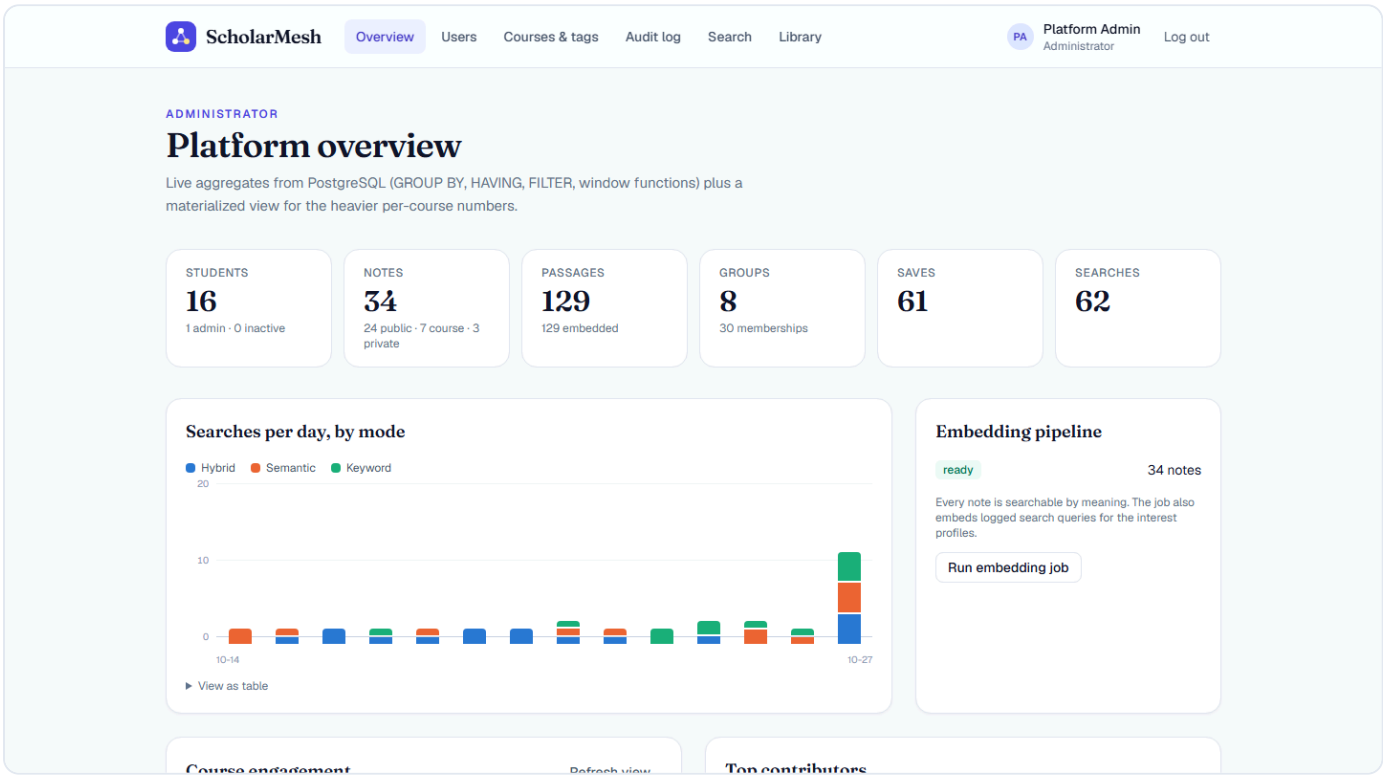
Study groups — best matches for you, your groups, and live seat counts.



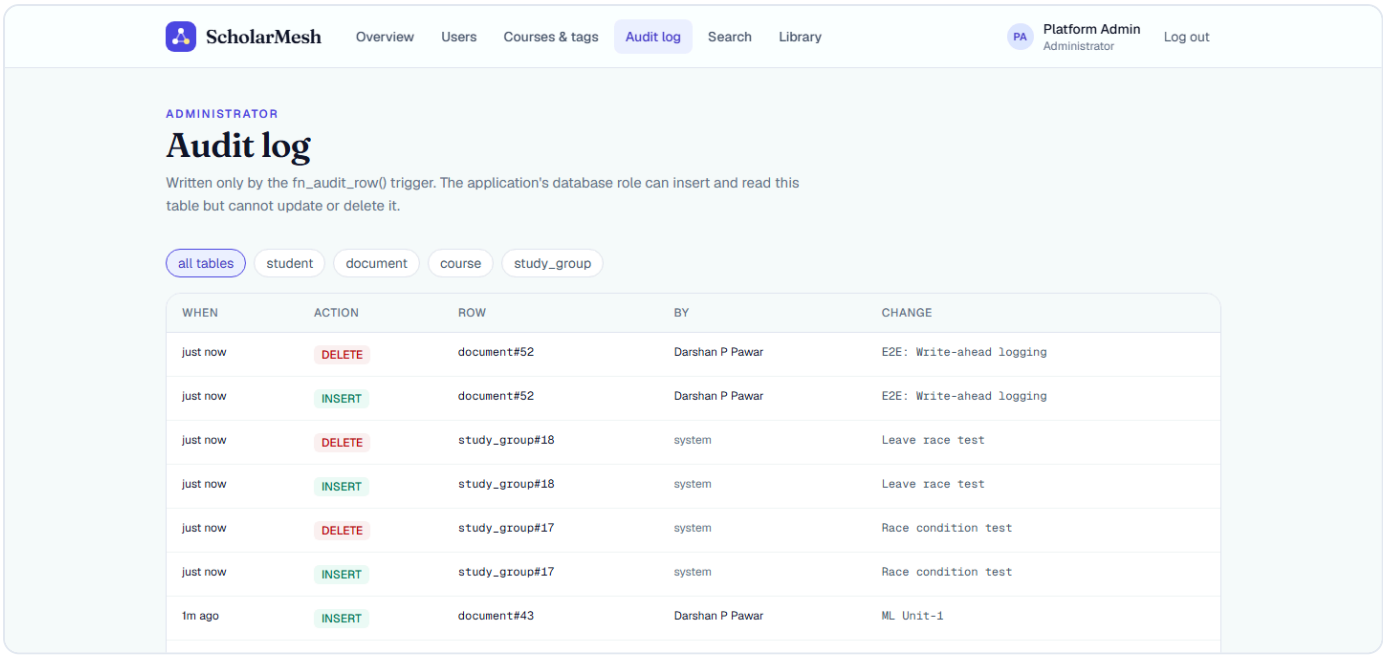
Uploading a note: file or pasted text, course, who can see it, tags.



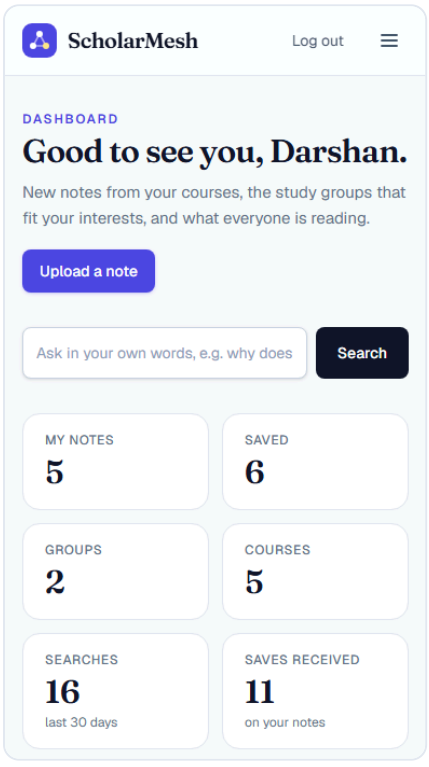
Joining a full group — the database itself refuses: "4 of 4 seats taken".



Admin dashboard — searches per day, notes per course, top contributors, popular questions and "content gaps" (questions with no notes yet).



Audit log — every important change, with who made it and when.



Works on phones too

The layout adapts to small screens; the menu folds into a ☰ button.

More screenshots

All 17 screenshots (login, library, courses, admin users, courses & tags …) are in `docs/screenshots/`.

5. Run it on your computer

Install these once (all free):

- **Docker Desktop** — docker.com/products/docker-desktop (runs the database)
- **Node.js 20 or newer** (LTS) — nodejs.org (runs the website)
- **Git** — git-scm.com (downloads the code)

You also need internet the first time (downloads ~1 GB of packages/images and a 23 MB AI model).

The commands below work in **PowerShell** (Windows) and in the **Terminal** on Mac/Linux.

1 Get the code

```
git clone https://github.com/Darshanpawar7/DBMS-Mini-Project.git
cd DBMS-Mini-Project
```

2 Start the database (Docker)

Open **Docker Desktop** first and wait until it says it is running. Then:

Set it up before on this computer? Skip the `cp` lines here and in step 3 — they would replace your existing settings, and the database still expects the old password.

```
cp .env.example .env          # database settings (example passwords are fine for local use)
docker compose up -d          # downloads and starts PostgreSQL + pgvector on port 5433
```

3 Set up the website

```
cd web
cp .env.example .env.local    # app settings (already match the database settings)
npm install                   # installs packages (a few minutes the first time)
```

4 Create the tables and load the demo data

```
npm run db:setup              # builds all tables, triggers, functions + demo data
npm run db:embed              # AI model reads every note (first run downloads the model)
```

5 Start the app

```
npm run dev
```

Open **http://localhost:3000** and log in:

Who	Email	Password
Darshan P Pawar (student)	`darshan.pawar@scholarmesh.test`	`Scholar@123`
Dhanush V Biradar (student)	`dhanush.biradar@scholarmesh.test`	`Scholar@123`
Admin	`admin@scholarmesh.test`	`Admin@12345`

Next time you only need: open Docker Desktop → in the project folder run `docker compose up -d` → then `cd web` and `npm run dev`. Stop the app with **Ctrl + C**; stop the database with `docker compose down` (your data is kept).

Sharing the computer or a network? Change the passwords in `.env`, put the same app password into `web/.env.local` (inside `DATABASE_URL`), and set `AUTH_SECRET` to a random value: `node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"`. Then run `npm run db:setup` again.

If something goes wrong

Problem	Fix
`docker` says it cannot connect	Docker Desktop is not running. Open it, wait a minute, try again.
"password authentication failed for user postgres"	The database was created earlier with another password (PostgreSQL keeps the first one), usually because `.env` was copied again. Put the old `.env` back, or start fresh — **this deletes the database data**: `docker compose down -v`, `docker compose up -d`, then steps 4–5.
"password authentication failed for user scholarmesh_app"	The app password in `web/.env.local` doesn't match `APP_DB_PASSWORD` in `.env`. Make them equal, then `npm run db:setup` (this also resets the demo data).
Port 3000 or 5433 already in use	App: `npm run dev -- -p 3001`. Database: change `POSTGRES_PORT` in `.env` and the port in `DATABASE_URL`, then `docker compose up -d`.
Semantic search finds nothing / notes show "Embedding…"	Run `npm run db:embed` (needs internet the first time to download the model).
Want the original demo data back	`npm run db:setup` then `npm run db:embed` (inside `web`).

6. Handy commands & where things are

Command (inside `web/` unless noted)	What it does
`npm run dev`	Start the website at http://localhost:3000
`npm test`	Run the 91 code + database tests
`npm run test:e2e`	Run the 10 browser tests (first time: `npx playwright install chromium`)
`npm run db:setup` / `npm run db:embed`	Rebuild the database / compute missing embeddings
(project folder) `cat db/10_important_queries.sql | docker exec -i scholarmesh-db psql -U postgres -d scholarmesh`	Run the demo SQL queries (joins, GROUP BY, window functions, vector search, EXPLAIN …)
(project folder) `docker exec -it scholarmesh-db psql -U postgres -d scholarmesh`	Open a SQL prompt inside the database

Folder / file	What's inside
`db/`	All SQL, run in order: `01` schema → `02` Phase 2 changes → `03` views → `04` functions → `05` triggers → `06` permissions → `07–09` demo data → `10` demo queries
`web/src/app/`	Website pages and the `/api` endpoints
`web/src/lib/repos/`	All SQL the app runs (parameterised, safe from SQL injection)
`web/tests/`	Automated tests
`docs/`	Technical report (PDF), this guide, diagrams, screenshots
`docker-compose.yml`, `.env.example`	Database container and settings templates

> **A 2-minute demo:** log in as Darshan → Search "*bcnf decomposition*" in Keyword mode (0 results), then Semantic mode (finds the note) → Study groups → open *Query Optimizers* → Join (the database refuses: group full) → log out, log in as admin → show the charts and the audit log.